

Don't see what you're looking for? Log in for the full Help Center experience

Authentication and authorization

Updated 2 years ago

Overview

This article describes various authentication flows in the Fireblocks platform.

REST API

Following are links to documentation in the Fireblocks developer portal for various actions you can take using the Fireblocks REST API.

- [Signing a request](#)
- [IP allowlisting](#)
- [Rate limiting](#)

Co-signer flows

This section outlines the authentication methods between co-signers, mobile applications, and the Fireblocks core infrastructure. In the MPC-CMP implementation:

- The CMP algorithm is used as part of both offline and online signing protocols.

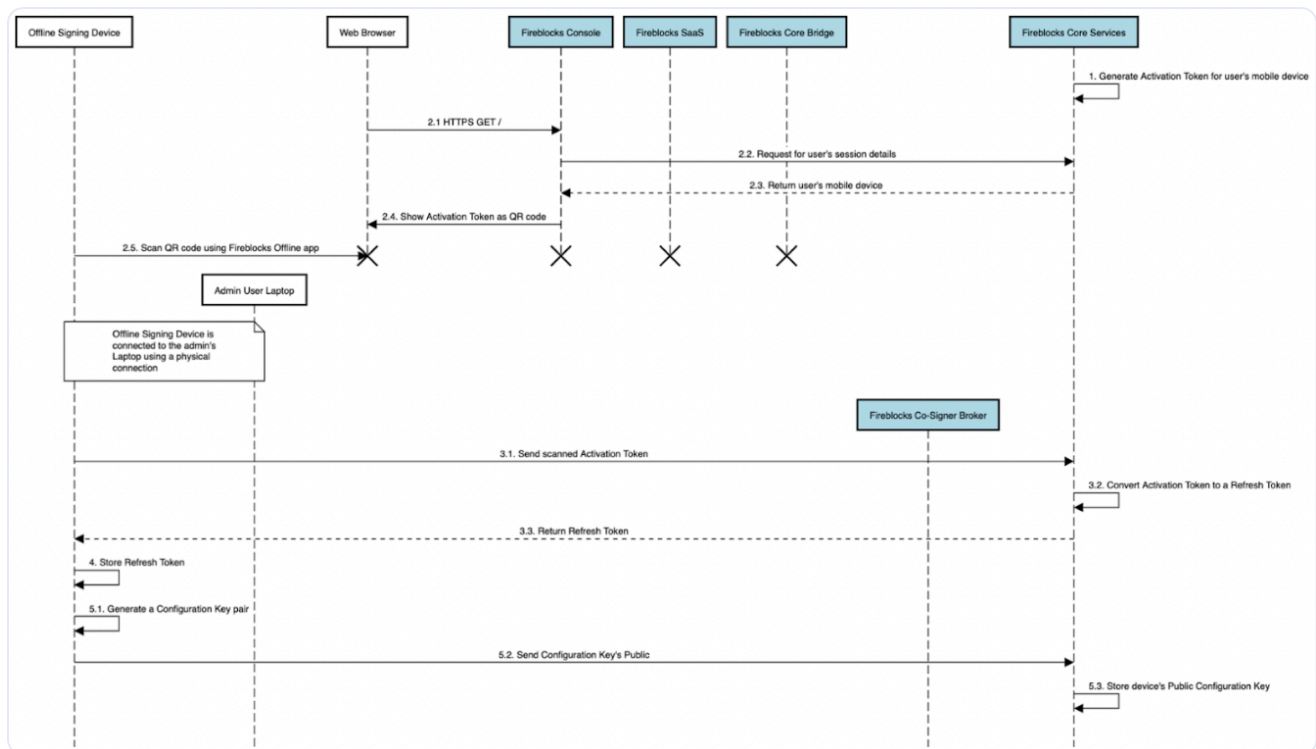
Aside from flow-specific assumptions written in the following subsections, all flows assume the following.

- The Fireblocks Core Services component is embedded with a key pair referred to as the *Root Key*. The Root Key acts as the main certificate authority to publish intermediate certificates to all Core Services.
- The Core Services make use of this Root Key pair to sign access tokens for various entities in the system.
- Customer components, including online and offline co-signers, are aware of the Core Services' public Root Key, whose correlating private key is used for signing

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/



Assumptions

1. The user has a valid access token by authenticating through the Fireblocks Console application on its web browser component.
2. Communication from Fireblocks SaaS to customer-hosted components is accomplished by the Fireblocks Core Bridge pulling requests from a message queue on the Fireblocks SaaS gateways.

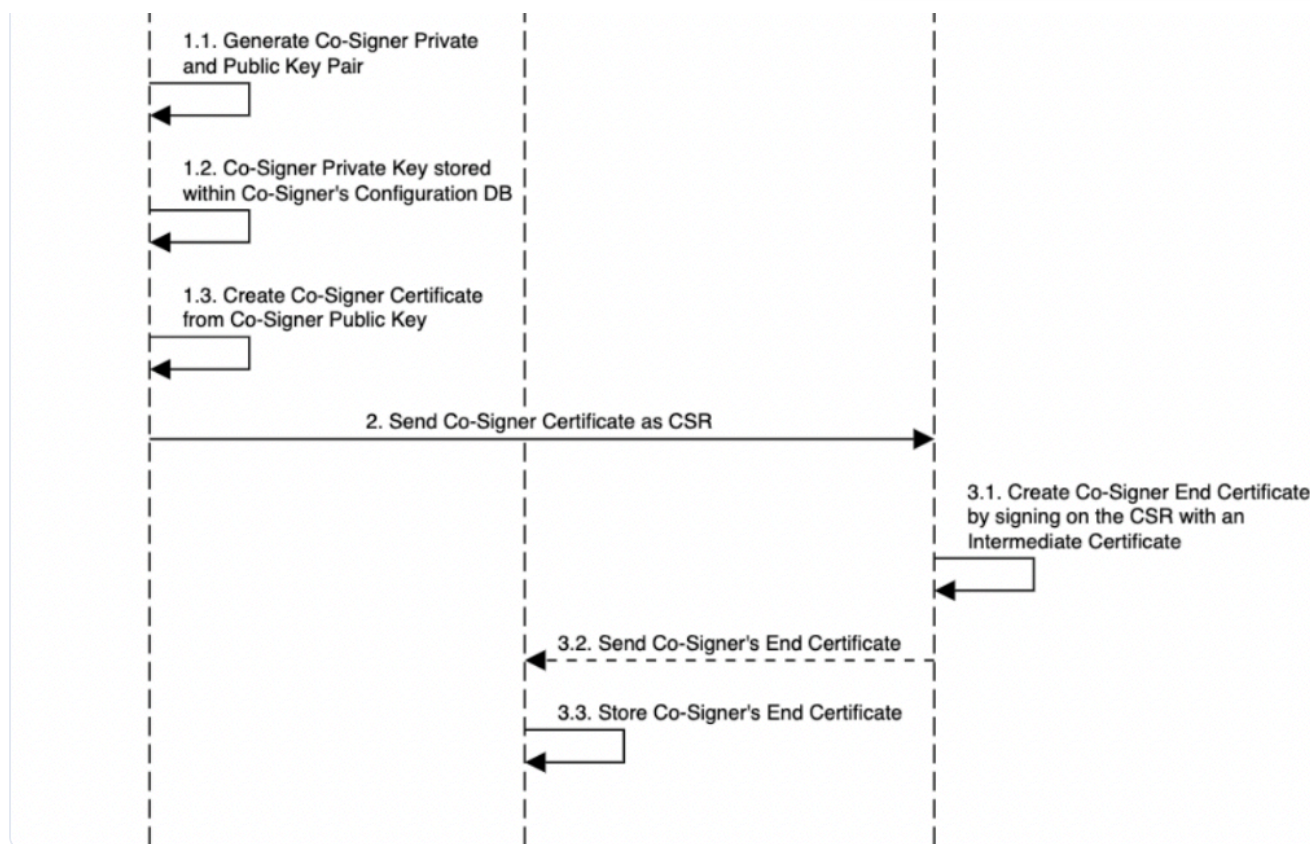
Flow

1. As part of the user creation process, Core Services generates and stores an activation token. The token is valid for seven days and configurable, if required.
2. When the user logs in for the first time, a QR code-containing token is displayed and is scanned using the Fireblocks mobile app.
3. After the user scans the QR code with the Fireblocks mobile app, the activation token is sent to the on-premises Core Services. The token is converted to a refresh token and sent back to the Fireblocks mobile app.
4. The Fireblocks mobile app receives the refresh token and stores it within KeyChain for future uses.
5. The Fireblocks mobile app generates a configuration key pair and sends the public key to Core Services for storage.

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help C

/



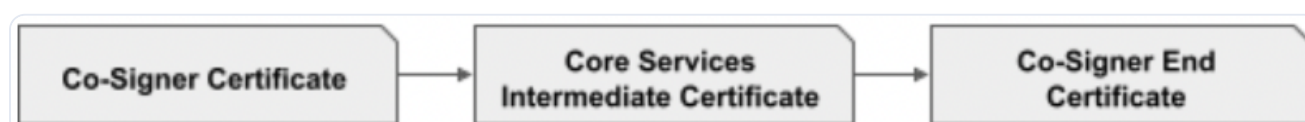
Assumptions

1. This setup phase is required once per co-signer instance and accounts for any key material provisioned on that instance.

Flow

1. Each co-signer generates a co-signer private and public key pair and creates a co-signer certificate from the generated public key. The generated private key is stored within the Configuration Database of the co-signer instance.
2. Each co-signer sends its CSR to the Core Services component through the Co-Signer Broker.
3. Fireblocks Core Services uses an intermediate certificate built into the image to sign the co-signer certificate, creating a co-signer end certificate for each co-signer. The end certificates are then sent and stored on the Co-Signer Broker for distribution across co-signers.

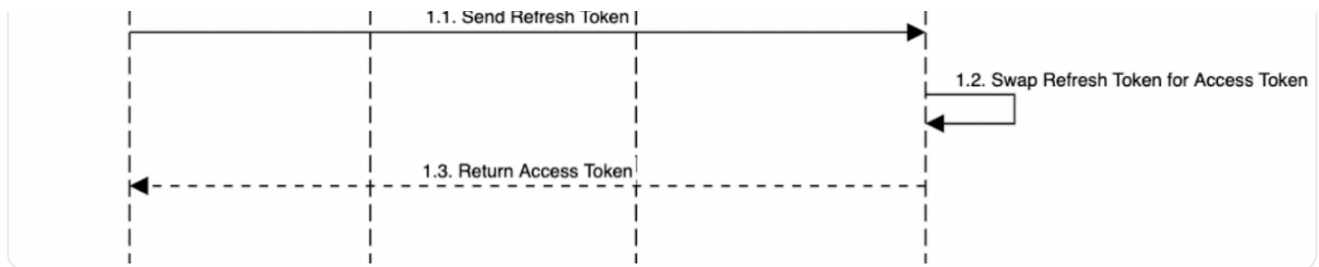
The flow above creates a chain of trust as seen here:



Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/



Assumptions

1. The offline signing device authentication setup and the co-signer authentication setup are completed successfully.
2. If preprocessing lasts more than 6 hours, this flow is re-initiated for the offline mobile app to get a new access token.
3. All CMP messages during the signatures' preprocessing are signed and verified by all co-signers.

Flow

1. The Fireblocks mobile app sends its refresh token to Fireblocks Core Services, where it is swapped for an access token (valid for six hours) and sent back to the mobile app.

References

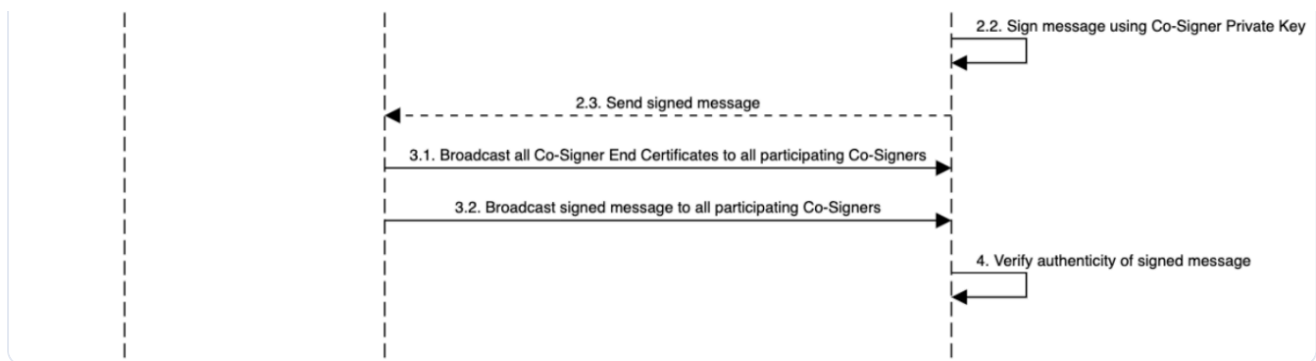
- [UC Non-Interactive, Proactive, Threshold ECDSA \(Ran Canetti and Nikolaos Makriyannis and Udi Peled\)](#)

Co-signer authentication

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/



Assumptions

1. The following flow assumes a signature threshold of 3/3 (3 participating co-signers).
2. Steps 2-4 in the flow below are executed multiple times according to the required rounds of communication as required by the MPC protocol.
3. For CMP signing, the third co-signer is fully offline. Therefore, it communicates with the Co-Signer Broker using QR codes via the Fireblocks Console as seen in the [offline signing device authentication](#) flow.

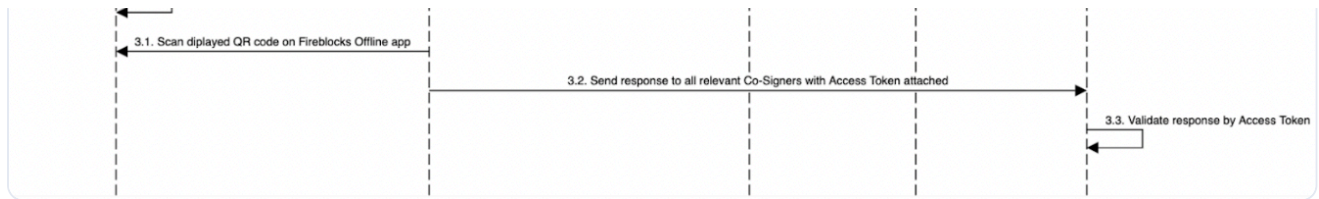
Flow

1. Fireblocks Core Services distributes (through the Co-Signer Broker) a command for all co-signers to start handling a transaction request for a signature.
2. Each co-signer creates an MPC message which is signed using its co-signer private key (stored in the Configuration Database) and sends the signed message to the Co-Signer Broker.
3. The Co-Signer Broker broadcasts all signed MPC messages and all end co-signer certificates to all co-signers.
4. Each co-signer validates signed MPC messages using the relevant end co-signer certificate and the constructed Chain of Trust.

Offline signing device authentication

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center



Assumptions

1. The following flow describes the authentication process for offline signers only and their correlating user entities within the Fireblocks environment.
2. A transaction was submitted for signing and is now pending for signature by one of the designated offline signers.

Flow

1. Once the user signs in to the Fireblocks Console, the browser authenticates the user by receiving an access token from Fireblocks Core Services.
2. The user uses one of his designated offline signing devices to scan the displayed QR code on the Fireblocks Console.
3. The offline Console scans the QR code from the offline signing device and sends it with the user's access token to the Co-Signer Broker for distribution between the co-signers. Co-signers validate the signature of the offline signer's QR code data and the access token.

Was this article helpful?

Yes

No

ON THIS PAGE

Overview

REST API

Co-signer flows

Don't see what you're looking for? Log in for the full Help Center experience